

→ Net system = multi-homogeneous processors  
 same task, diff. data  
 diff. task

→ System bus needs to be scheduled

→ Networked System

we need to know the identity of the other to connect  
 need to know the identity of the other to connect  
 need to know the identity of the other to connect

→ Distributed System

ind., heterogeneous systems, not networked or perform tasks on  
 need to obtain data from a loc. / system without knowing its identity

Virtual System / Interface / Image / Computer View

enables transparency (need not know where the data is, how it is stored, etc.)

multidatabase != distributed database env.

homogeneous model to store data (same schema)

heterogeneous model (different storage patterns)

common this course

Database Centralized System

simpler issues

Clients on same / diff. site

DB duplicate → diff. site

all updates

DB

Server on same / diff. site

duplication to avoid access issues

Availability \* ↓ cost better performance

issue → maintaining these mirrors

Key modules of Centralized RDBMS Software = ?

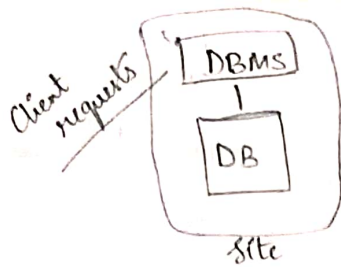
SQL → DDL → Create the defn. of the database  
 DML  
 Data Control Lang.

DDL compiler  
 DML compiler

Query Processing Component  
 Query Optimizing Component

DB transaction → Scheduler, Concurrency Controller  
 Reorder

should follow ACID properties



## Parallel Systems

→ improving performance

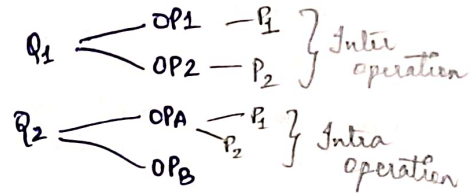
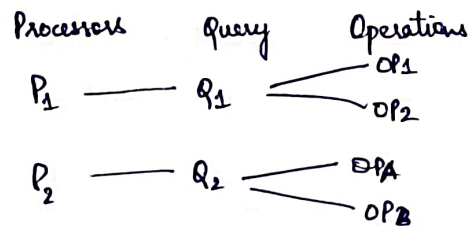
- Speedup
- Scale up
- Throughput

Parallelism in DB Systems

- in general
  - More hardware + same problem
  - More hardware + larger problem
- based on how to divide the work?

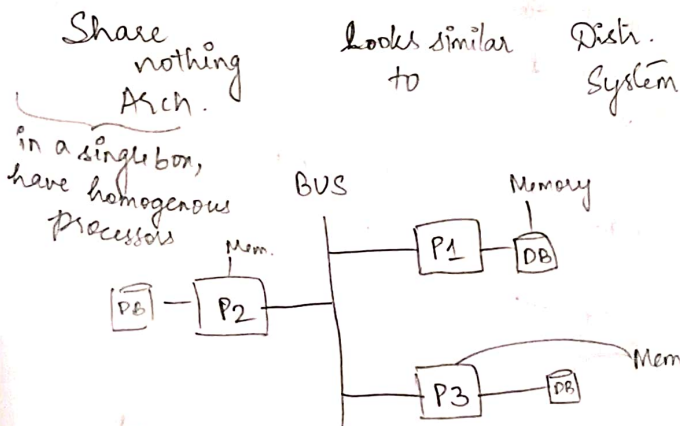


Inter query  
Wism



Why do we need parallelism?  
eg. Sorting a large table

These exist naturally in dist. system



Bus is usually the prob.  
Bus is used only for synchr.

Q. What are the other issues of parallel system? bus?

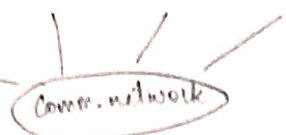
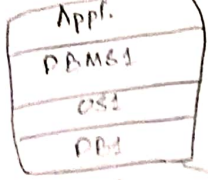
## Distributed Database Systems

→ to support decentralized orgs.

using client-server setup for every pair of client-site is expensive  
Need a way to access conditionally

Multiple logically related databases, located @ diff. locations, communicating through a network





- \* Independently capable to manage own data, fully autonomous
- \* Each site is heterogeneous
- \* Data is logically related → all sites can have same DBMS software
- \* Connected by a highly unreliable comm. network, communicate ONLY by MESSAGE PASSING
- we need to maintain the integrity of the (system) data, ... } all ACID properties need to be satisfied
- can become homogeneous sites
- common schema/structure (similar constraints, keys, etc.)

User must perceive the entire org's system to be a single entity

- user: accessing data of site<sub>i</sub> } local queries — present in client-server db model
- user: site<sub>j</sub> } remote queries — needs query forwarding
- user: accessing multiple sites (can include site<sub>i</sub>) } global queries — unique to distributed db model

Maintaining Data integrity  
↓  
harder / bigger issue in distributed database cases

How does the system know whether a query is local / remote or whether it has the reqd. data or not?

↳ Data dictionary / metadata  
- has details abt. the db

↳ contents = ?  
→ each site maintains info. about its own db  
need to know which sites have the data, how to process it

- find the target sites
- divide the query into subtransactions, send each subst. to the reqd. sites
- these sites process the subqueries
- return the results to originating site
- combine these results

On each site, on top of local DBMS, an extra module has to be added to handle above situations

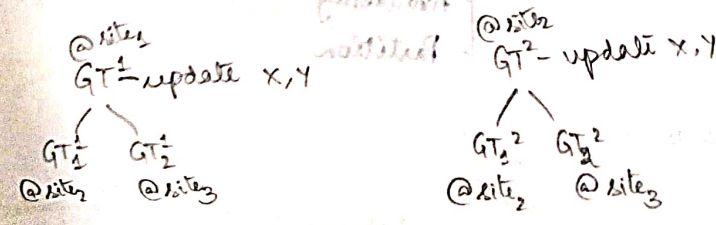
- ② Distributed Query Processing
- ↳ need to have "Global Data Dictionary"
- Process Optimiz.

to know what other sites have

Just by checking the timestamps, we cannot decide which T has to be done first as they have their own local clocks

need "TOTAL ordering" of T

No global clock } communicate only through msg. passing  
No global memory }  
Concurrency control  
Recovery / Reliability



If only one of GT<sub>1</sub><sup>2</sup> & GT<sub>1</sub><sup>3</sup> succeed, then it should not be committed as they are parts of GT<sup>1</sup>.

③ Distributed Transaction

Does user know which kind of query is being sent? If yes, should he decide where it should be sent, how? should he split, etc? NO.

Job of DBMS

User should see a "single virtual DB System view"  
Should take care of all transparency for.

User must be relieved from these technicalities.

\* Data is fragmented

\* Need to take care of availability of data when 1 site fails & replicated

DBMS must take care of:

**Distribution Transparency**

- Fragmentation Transparency
- Replication Transparency
- Naming Transparency

DB should know which "replica" the data is taken from

Degree of replication depends on business strategy

**I Distributed DB design**

- Frag. design
  - Allocation design
  - Repl. design
- how to distribute the data

Modeling may reduce & included for exam  
GFS, HDFS & File Systems  
NoSQL

BLOBs - Binary Large Objects

Can unstructured data be stored in SQL / similar DBs?

DB  
Relational

NoSQL

No common interface like SQL, but has:

\* to maintain data integrity for unstructured data  
eg: MongoDB

Document Stores (JSON, XML)

No query optimization  
Graph Stores

\* Schema-less DB

flexibility

\* Horizontal scalability

\* High availability - as data is stored in at least 3 places (replication)

Not strict ACID

\* BASE

Basically Available

Eventual Consistency

(not immediate)

CAP Theorem (Brewer's Theorem)

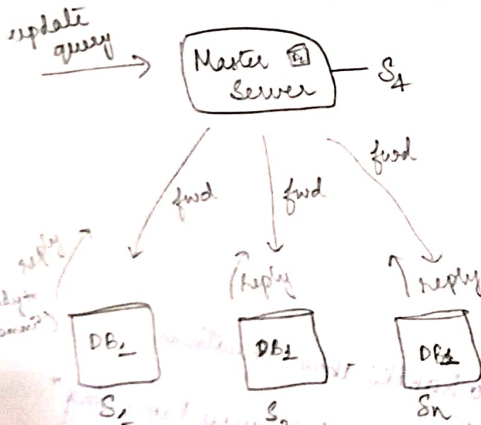
- Consistency
- Availability
- Partition

eventual consistency

See Study:

Walmart's Data Structure

2-phase protocol (msg. passing)



Election Alg. → To choose 1/many master servers/nodes

Alternate method:

\* Master updates

\* Other replicas update slowly, taking 1 update at a time from their update queues in sequence

eventual consistency



Codd's Rules (These must be satisfied by a DBMS for it to be called as a DDBMS)

\* Every system must be autonomous

\*

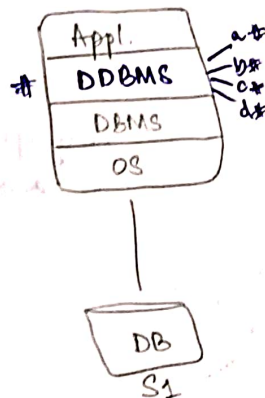
Distribution Transparencies  
 → Location  
 → Fragmentation  
 → Duplication

DBMS

Commercial DDBMS

do not satisfy all 12 Codd's Rules

Distributed Env.



specific lang. to access/update

- Key: doc  
 key: doc

# → addl. modules needed in the distr. case

help to take independent decisions

Knowledge of data in the form of data dictionary

b # Distributed Query Processor & Optimizer (DQP & O)

c # Dist. Transaction Mgmt.  
 d # Dist. Recovery Mgmt.

Local  
 Global  
 a # (GDD)  
 → keep in single loc.  
 OR  
 partition & store in multiple places

Placement of Data  
 - Appl. prog.  
 - DDBMS SW

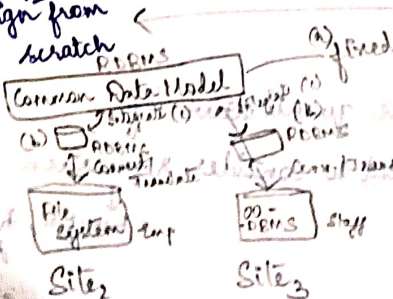
Data verified after analysis based on summaries

## Distributed DB Design

Placement of data across various sites

Dist. Systems

Design for ex. that already exists, but give a distributed view  
 Design from scratch



All data in Common Data Model should be complete & concise.

Integration approach

Design approaches

Bottom-up  
 Top-down  
 from requirements to design

Data Integration  
 - Physical  
 - Logical  
 - or hybrid (hybrid approach)

a) fix a common model  
 b) Convert / Translate existing model to DDBMS  
 c) Integrate to global data  
 lot of rules needed for mapping, resolving constraints, etc. (e.g. Emp.)  
 - Highly summarized  
 - Aggregated  
 - Highly denormalized

Top Down

Analysis  
(freq. of accesses,  
who accesses, etc.)

Info 1

Access-pattern  
behavior

Static - this course  
Dynamic

Access Pattern  
behavior

Qualitative (kind of access,  
which attributes)  
Quantitative (freq. of queries,  
no. of reads/writes)

80:20 rule

80% of access pattern can be captured by 20% of the behavior

Info 2

Design the  
Global Conceptual  
Schema

(can be partial,  
update later)

Req. Colln.  
↓  
Global Conceptual  
Schema  
↓  
Frags.  
↓  
Alloc.

Distributed DB  
Design

Fragmentation design = logical division

Allocation design = physical mapping to the sites (considering site constraints)

Replication design  
why? For availability

## A) Fragmentation design

- Use qualitative info.

- what is accessed
- type of data accessed
- how it is accessed? using what predicates?

Every operation needs only  
a subset of the data

- Why to fragment? why not replicating the entire DB?

- Locality of reference ↑ the LoR of most of the queries
- Load distribution to avoid requiring data from another/multiple other sites
- Security
- Performance
- Efficiency
- Availability

need to decide which  
tuples, which  
attributes, etc.  
are to be put on  
every site

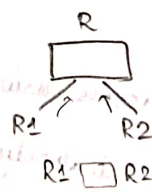
- How to fragment?

Horizontal  
Vertical  
Hybrid

- How much to fragment?

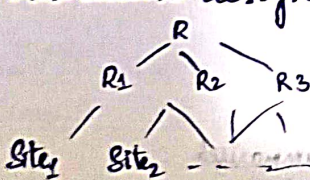
based on "Qualitative Info."

- How to test the correctness of the fragments?



find an operator that can  
merge these correctly

## B) Allocation design



logical  
frag.

allocation = physical  
mapping

- Use quantitative info.

- Need to consider replication decisions as well

benefits, how near should the  
duplicates be to ↑ loc. of  
ref. by  
avail., etc.



DB Requirements

↓  
Global Conceptual Schema

Multiple steps like ER modelling.

Access Behavior

done automatically by cloud providers

Design can be done at  
 - Design Time - Enterprise DBs — this course  
 - Run Time - Cloud Service Providers

Common Mean Algorithm

Divide any global relation  $R \Rightarrow (R_1, R_2, \dots, R_n)$  horizontally, i.e., tuples are distributed, schema for all remains same (Primary Key, Foreign Keys remain intact)

① Pick up the queries that are applied only on  $R$   
 ↓  
 Collect the predicates on the attributes of  $R$  (say  $P_1, P_2, P_3$ )

② Combine the predicates to form minterm predicates (in both original form & negated form)

$P_1, P_2 \quad P_1 \wedge P_2 \quad \neg P_1 \wedge P_2 \quad P_1 \wedge \neg P_2 \quad \neg P_1 \wedge \neg P_2$

③ Assess these combinations & retain those that are relevant (make sense)  
 Remove the irrelevant ones

A) Fragmentation Design

a) Horizontal F  
 - Primary  
 - Derived  
 (mainly for joins)

For each  
 ④ Remaining relevant minterm predicate, decide & reserve fragments for each.

eg Employee (Empno, Sal, Loc, Dno)

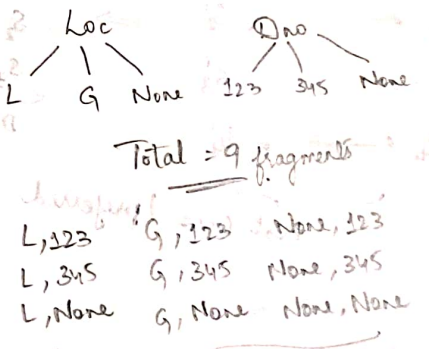
4 Predicates  $\Rightarrow 2^4$  combinations

$P_1$  Loc = 'London'     $P_2$  Loc = 'Geneva'     $P_3$  Dno = 123  
 $P_4$  Dno = 345

$P_1 \wedge P_2 \wedge P_3 \wedge P_4 \rightarrow$  Loc & Dno cannot have 2 values  $\Rightarrow$  Irrelevant

Meaningful ones:

$P_1 \wedge \neg P_2 \wedge P_3 \wedge \neg P_4 \rightarrow$  Loc = London, Dno = 123  
 $\neg P_1 \wedge P_2 \wedge P_3 \wedge \neg P_4 \rightarrow$  Loc = Geneva, Dno = 123  
 $P_1 \wedge \neg P_2 \wedge \neg P_3 \wedge P_4 \rightarrow$  Loc = London, Dno = 345  
 $\neg P_1 \wedge P_2 \wedge \neg P_3 \wedge P_4 \rightarrow$  Loc = Geneva, Dno = 345  
 $\neg P_1 \wedge \neg P_2 \wedge \neg P_3 \wedge P_4 \rightarrow$  Loc = neither of London, Geneva



Total = 9 fragments

L, 123    G, 123    None, 123  
 L, 345    G, 345    None, 345  
 L, None    G, None    None, None

|      |         |
|------|---------|
| None | 123+345 |
| L    | 123+345 |
| G    | 123+345 |
| L+G  | 123     |
| L+G  | 345     |
| L+G  | 123+345 |
| L+G  | None    |

Irrelevant.

Union of all of these gives R  
 Reconstruction

Correctness Check: (each frag. must satisfy these)

- \* Completeness
- \* Reconstruction / Correctness - need to find a suitable operator
- \* Disjointness

eg Employee (Empno, Empname, Sal, Loc, Dno)  
 Project (Dno, Pno, Hours)

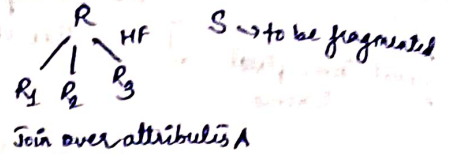
And employee details, Pno of emp. in Loc: 'Geneva'.

Join of these 2 is computed freq. for such queries  $\rightarrow$  Join is precomputed & stored  
 ↓  
 Derived Horiz. Frag.

Derived HF  $\rightarrow$  Aim: To  $\downarrow$  the comm. cost

(Use the fragments, create joins & store the resultant tables)

Use the fragments of another relation to fragment a relation



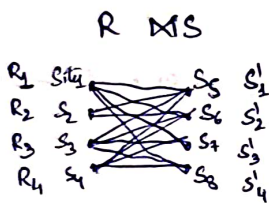
(frag. of  $R$ )  $\bowtie_A S$  } this would give attributes of  $R$  as well

So, what we do is:

$S \bowtie_A (\text{frag. of } R) \rightarrow$  perform the join, but project the attributes of  $S$  only

$$\Pi_S (S \bowtie_A \sigma_{P_1}(R))$$

project the attr. of  $S$       frag. of  $R$



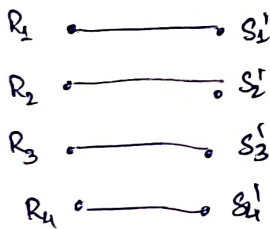
need to get data from both  $R$  &  $S$

$R_1$  sent to  $S_1, S_2, S_3, S_4$   
 $R_2$  - - -  
 $R_3$  - - -  
 $R_4$  - - -

a lot of communication  
 ( $\uparrow$  data has to be sent)

too much comm.

$\downarrow$   
 $\therefore$  avoided



$$\begin{cases} S_1 \leftarrow S \bowtie_A R_1 \\ S_2 \leftarrow S \bowtie_A R_2 \\ S_3 \leftarrow S \bowtie_A R_3 \end{cases}$$

DHF of  $S$  hence obtained

$$\text{Final } S = S_1 \cup S_2 \cup S_3$$

best case / preferred

Q.  $S$

| A | B | C    |
|---|---|------|
| x | y | 1    |
| y | z | 2    |
| z | y | NULL |

$R$

| C | D |
|---|---|
| 1 | 3 |
| 2 | 4 |
| 3 | 5 |

$S \bowtie R$

| A    | B    | C    | D    |
|------|------|------|------|
| x    | y    | 1    | 3    |
| y    | z    | 2    | 4    |
| z    | y    | NULL | NULL |
| NULL | NULL | 3    | 5    |

Q. Can foreign key have duplicate values? (2-tuples having same value in FK col.)

Yes



## b) Vertical Fragmentation

mainly for using a subset of attributes

$R^{PK}(A, B, C, D)$

$R_1(A, B)$

$R_2(A, C)$

$R_3(A, D)$

PK needs to be duplicated in all frag. so that R can be obtained back again

$S(1 \dots m)$

$m \rightarrow$  non PK attributes

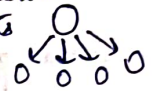
Worst case, no. of fragments = "Bell number" =  $m^m$

Methods to reduce this

Grouping  $\rightarrow$  combine some attributes to form groups (1 param)

Splitting  $\rightarrow$  based on app's behavior

Split the entire attribute set into subsets



## Heuristics from Access Behaviour

1. Attribute usage
2. Attribute accessed together
3. Frequency of access
4. Query originating site

points to keep in mind to help using vertical fragmentation

### 1. Attribute usage

| Query | $q_1$ | $q_2$ | $q_3$ | $q_4$ |
|-------|-------|-------|-------|-------|
| $S_1$ | 20    |       |       |       |
| $S_2$ | 40    |       |       |       |
| $S_3$ | 30    | time  |       |       |

$A_1, A_2, A_3, A_4, A_5, \text{non-PK attr.}$

|       | $A_1$ | $A_2$ | $A_3$ | $A_4$ | $A_5$ |
|-------|-------|-------|-------|-------|-------|
| $q_1$ | 1     | 0     | 0     | 1     | 1     |
| $q_2$ | 0     | 1     | 1     | 0     | 1     |
| $q_3$ | 0     | 1     | 0     | 1     | 0     |
| $q_4$ | 1     | 1     | 1     | 1     | 1     |

$\rightarrow$  if  $q_i$  needs attr.  $R_1$  0 else

originating site

$\rightarrow$  can capture the freq. of usage of queries by appl.

Type of access  $\rightarrow$  does not impact fragmentation } needed in "Allocation Design" stage (Read/Write)

Frag. depends on origin of query, loc. of reqd. data, etc.

From the no. of times each pair  $A_i, A_j$  are accessed together, we can create an

Attribute affinity matrix  $\rightarrow$  Symmetric matrix Affinity b/w  $A_1, A_2$  (attributes)

- $\rightarrow$  Find queries that access both together
- $\rightarrow$  Find freq. of those queries

## "Bond-Energy Algorithm"

Get Info:

- $\rightarrow$  freq. of queries
- $\rightarrow$  origin of query

### 2. Calculate Attribute Affinity Matrix

$$aff(A_i, A_j) = \sum_{\text{app. sites}} (\text{all queries}) \text{ query accesses that access } A_i, A_j \text{ together}$$

|       | $S_1$ | $S_2$ | $S_3$ | $S_4$ | Sum |
|-------|-------|-------|-------|-------|-----|
| $q_1$ | 20    | 10    | 0     | 0     | 40  |
| $q_2$ | 40    | 20    | 10    | 0     | 70  |
| $q_3$ | 10    | 20    | 0     | 0     | 30  |
| $q_4$ | 10    | 10    | 10    | 10    | 40  |

optimise based on app's requirement

$A_1, A_2$ : only  $q_1$   
 $S_1, A_1, A_2$ : freq. ( $q_1$ )

Rearrange / Reorganize the (table) s.t values come together (all together, 0s tog.)

|       | $A_1$ | $A_2$ | $A_3$ |
|-------|-------|-------|-------|
| $A_1$ |       |       |       |
| $A_2$ |       |       |       |
| $A_3$ |       |       |       |

Each block / square  $\downarrow$  1 fragment!

### 3. Bond Energy Algorithm - for clustering the attributes

Uses bond values to maximise the global affinity sum / affinity of it with its neighbors is maximised & rearrange the  $A_i$  values accordingly

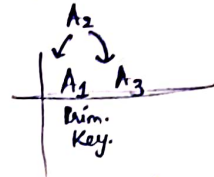
$(A_i, A_k, A_j)$

↑  
placement of  
 $A_k$  b/w  $A_i, A_j$  based on

$Contn(A_i, A_k, A_j) = Bond(A_i, A_k) + Bond(A_k, A_j)$

-  $Bond(A_i, A_j)$

If global aff. after  $A_k$ 's placement ↑, keep  $A_k$  b/w  $A_i, A_j$ .  
Otherwise, find another place.



Primary key should be in all vertical fragments  
attr.

Contr.  $(0, A_2, A_1)$  v/s contr.  $(A_1, A_2, A_3)$

(both cols. & rows)

After comparing all such contr. & reordering,  
natural clusters will be formed.

Create groups of attr. based on application's requirements  
(or divide based on applications)

Like Appl. 1 uses  $A_1, A_2, A_5$ , & so

gp. them together

|       | $A_1$ | $A_2$ | $A_5$ | $A_6$ | $A_7$ |
|-------|-------|-------|-------|-------|-------|
| $A_1$ |       |       |       |       |       |
| $A_2$ |       |       |       |       |       |
| $A_5$ |       |       |       |       |       |
| $A_6$ |       |       |       |       |       |
| $A_7$ |       |       |       |       |       |

## c) Hybrid Fragmentation

Primary HF

| row    | $A_1$ | $A_2$ | $A_3$ | $A_5$ | $A_4$ |
|--------|-------|-------|-------|-------|-------|
| row 1  |       |       |       |       |       |
| row 2  |       |       |       |       |       |
| row 3  |       |       |       |       |       |
| row 4  |       |       |       |       |       |
| row 5  |       |       |       |       |       |
| row 6  |       |       |       |       |       |
| row 7  |       |       |       |       |       |
| row 8  |       |       |       |       |       |
| row 9  |       |       |       |       |       |
| row 10 |       |       |       |       |       |

Some appl. need not require all A's.

So, perform "Vertical Frag." on this  
Primary HF.

## Combining Fragments

$R_1, R_2, \dots, R_n \xrightarrow{\text{using Relational Algebra Operator}} R$

## B) Allocation Design

NP  
Hard  
Problem

Use heuristics  
based on the  
info. collected

Frag.  $F = \{F_1, F_2, \dots, F_n\}$

Sites  $S = \{S_1, S_2, \dots, S_k\}$

Appl.  $A = \{A_1, A_2, \dots, A_m\}$

↓  
provide  
Info.

Physical Mapping - Optional  
distr. of  
F to S

$F_i \rightarrow S_j$

or

$F_i \rightarrow S_j, S_k$

\* Appl. info (no., which appl. access which frag.)

\* Frag. schema

\* How frag. were formed

\* Site info (like capacity)

Interconnections, etc.  
(so that we can  
get  $F_i$  stored  
elsewhere,  
and understand time

\* Network info

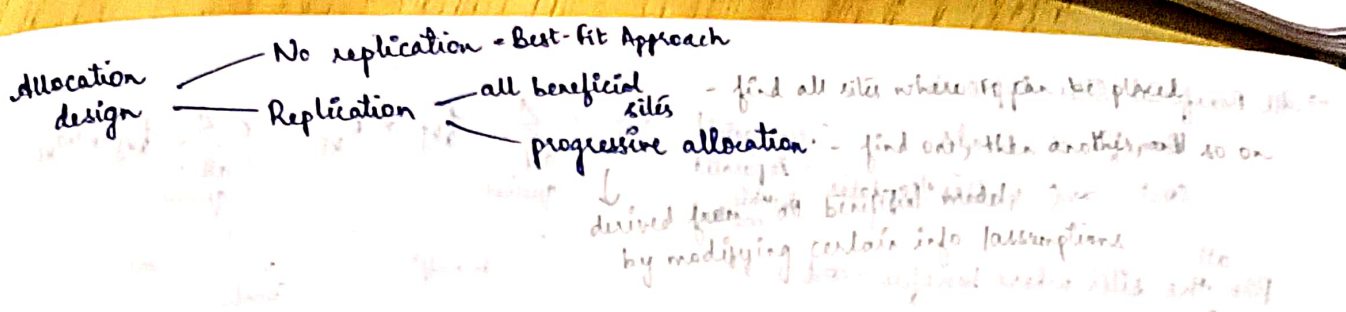
bandwidth

\* Type of query access (read / write)

\* Query, originating loc. of query

If read ↑, ↑ replicas will help  
write ↑, ↓ replicas will help





Optimize the Total Cost = A+B+C

Query Processing Cost

Storage Cost

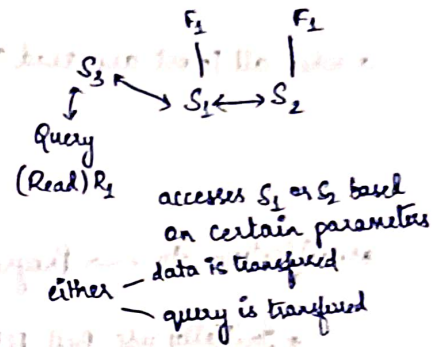
varies based on system used

Processing Cost

Communication Cost

main players in the Total Cost

Read Write



eg.  $S_3$  broadcasts  $R_1$ , and the response of whichever sites responds first is taken

eg.  $w_1$  (write) query  $\rightarrow$  both  $S_1, S_2$  need to do it  
extra cost (as all sites where  $F_1$  is present need to be updated)

to ensure consistency!

Benefit  $\rightarrow$  for read operation (any 1 site is enough)  
Disadv.  $\rightarrow$  for write operations ( $\uparrow$  costs as all sites need be updated)

$\rightarrow$  Best Fit Approach

Fragments - i

Sites - j

Appl. - k

B = benefit

try to maximize no. of local reads & writes

$$\text{Benefit of } B_{ij} = \sum_{\text{all queries}} n_{ki} \times \frac{n_{ki}}{n_{ki} + u_{ki}}$$

$n_{ki}$  = no. of read op + update op.  
 $u_{ki}$  = no. of update op. x freq.  
 $n_{ki}$  = no. of read queries x freq.

| Transaction    | Sites           | Types of accesses                | Freq.               |
|----------------|-----------------|----------------------------------|---------------------|
| $T_1$ or $T_3$ | $S_1, S_3$      | 2R, 1W (on $F_1$ ), 2R ( $F_2$ ) | $R_f = 10, W_f = 5$ |
| $T_2$          | $S_1, S_2, S_4$ | 3R, 1W ( $F_1$ ), 2W ( $F_2$ )   | $R_f = 10, W_f = 5$ |
| $T_3$          | $S_2, S_4$      | 2R ( $F_1$ ), 2W ( $F_2$ )       | $R_f = 10, W_f = 5$ |

check only local

|       | $T_1$         | $T_2$         | $T_3$  | Total Local Op. |
|-------|---------------|---------------|--------|-----------------|
| $F_1$ |               |               |        |                 |
| $S_1$ | 2R(10), 1W(5) | 3R(10), 1W(5) | 0      | 60              |
| $S_2$ | 0             | 3R(10), 1W(5) | 2R(10) | 55              |
| $S_3$ | 2R(10), 1W(5) | 0             | 0      | 25              |
| $S_4$ | 0             | 3R(10), 1W(5) | 2R(10) | 55              |
| $F_2$ |               |               |        |                 |
| $S_1$ | 2R(10)        | 2W(5)         | 0      | 30              |
| $S_2$ | 0             | 2W(5)         | 2W(5)  | 20              |
| $S_3$ | 2R(10)        | 0             | 0      | 20              |
| $S_4$ | 0             | 2W(5)         | 2W(5)  | 20              |

$\rightarrow$  Select the max!

$\frac{25}{35}$   
 $\frac{60}{60}$

Now, we should whether the chosen sites can accommodate the fragments assigned. If they do not fit, choose the next best site.

These models Do Not Consider:  $\rightarrow$  Concurrency Control Cost  
(No repl. models? All 3 models?)  $\rightarrow$  Intensity Maintenance Cost  
Just consider type of accesses & freq. of each

## → All Beneficial Sites Approach

Benefit  $\propto$  Reads

Cost  $\propto$  Writes/Updates

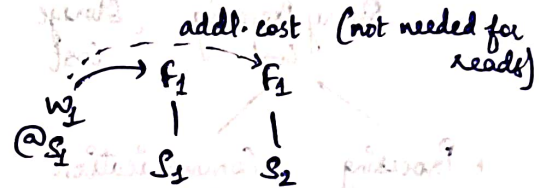
segregated/  
separated  
in this  
method

all  
Pick the sites where benefits > cost

$$B_{ij} = \underbrace{\sum_{\text{all queries}} f_{kj} \times R_{ki}}_{\text{benefit}} - \underbrace{c \sum_{\text{all queries}} f_{kj} \times w_{ki}}_{\text{cost}}$$

Const.

When all / most are read accesses, go for the 'all beneficial sites' approach



## Modification to → Progressive Allocation

1st iteration

\* Initially use Best Fit / All Benef. approach to choose the best site & place the fragment (ONLY 1)

Later iteration

\* Let that fragment be in that site, and then check the other sites for suitability (retain it)